

PLAN MAESTRO PARA CODEX

Aplicación de Finanzas Personales - Web + Mobile

Proyecto nuevo e independiente

Objetivo del documento

Servir como prompt y especificación de trabajo para ChatGPT/Codex, de modo que pueda planificar, construir, probar y documentar una plataforma financiera personal moderna, segura, automatizada y escalable sin depender de proyectos previos.

Modalidad	Definición
Proyecto	Nuevo, independiente, repositorio nuevo
Plataformas	Web, PWA, Android e iOS
Enfoque	Offline-first + automatización + analítica + seguridad
Público inicial	Uso personal / beta privada
Prioridad	Seguridad y confiabilidad de datos financieros

1. Instrucción principal para Codex

IMPORTANTE: este trabajo debe iniciar como un proyecto completamente nuevo. No modificar, importar ni reutilizar automáticamente GASTOSPE, app-gastospe, FDEBT, FDEBTV2 ni ningún repositorio previo. Si alguna idea anterior resulta útil, primero documentarla y pedir que se replique de forma limpia dentro de la nueva arquitectura.

2. Objetivo general

Diseñar y construir una plataforma de finanzas personales multiplataforma que centralice gastos, ingresos, cuentas, tarjetas, deudas, presupuestos, metas de ahorro, suscripciones y análisis financiero; que pueda incorporar movimientos automáticamente desde fuentes autorizadas como Gmail; y que ofrezca una experiencia minimalista, premium, amigable y segura tanto en web como en dispositivos móviles.

3. Objetivos específicos

- Crear una fuente única y confiable de información financiera personal.
- Reducir al mínimo el registro manual mediante automatizaciones autorizadas.
- Permitir trabajo offline y sincronización segura al recuperar conectividad.
- Ofrecer dashboards claros, filtros, búsquedas y proyecciones financieras.
- Construir una experiencia nativa y cómoda en Android/iOS, además de una web potente.
- Aplicar seguridad por diseño, aislamiento por usuario, auditoría y gestión segura de secretos.
- Definir un producto escalable, modular y mantenible que pueda crecer por versiones.

4. Alcance funcional

Módulo	Alcance
Dashboard	Saldo total, ingresos, egresos, balance, ahorro, flujo de caja, próximos pagos y alertas.
Movimientos	Gastos, ingresos, transferencias, pagos, reembolsos y ajustes.
Cuentas	Ahorros, corriente, efectivo, billeteras digitales, inversiones y otras.
Tarjetas	Línea, disponible, consumo, fecha de corte, fecha de pago, cuotas y utilización.
Deudas	Capital pendiente, interés, cuotas, simulación de adelantos y amortizaciones.
Presupuestos	Límites mensuales por categoría y alertas progresivas.
Metas	Objetivos de ahorro y cálculo de aportes

	semanales/quincenales/mensuales.
Suscripciones	Detección de pagos recurrentes, próximo cobro y costo anual.
Analítica	Comparaciones mensuales, tendencias, proyección de gasto y explicaciones.
Automatización	Ingreso de movimientos por Gmail y otros conectores autorizados.
Asistente IA	Consultas naturales sobre información financiera del propio usuario.
Exportación	CSV, Excel y PDF por rango, cuenta, tarjeta o categoría.

5. Principios de producto

- Mobile-first sin degradar la experiencia desktop.
- Offline-first para operaciones esenciales.
- Automatización asistida, nunca opaca.
- Toda recomendación debe poder explicar qué datos la originaron.
- Nunca almacenar credenciales bancarias, PIN, CVV ni secretos en el cliente.
- Diseño minimalista: mostrar primero lo importante y revelar detalle bajo demanda.
- El usuario debe poder exportar, eliminar y revocar sus datos e integraciones.

6. Estrategia tecnológica a evaluar

Codex debe comparar técnicamente las siguientes alternativas antes de implementar el producto completo:

- Angular para Web + Flutter para Android/iOS + backend compartido.
- Angular + Ionic/Capacitor para Web/PWA/Mobile.
- Flutter para Web + Android + iOS con una única base principal.

Criterios de decisión: rendimiento, UX móvil, reutilización de código, soporte offline, biometría, notificaciones, mantenibilidad, seguridad, costos y velocidad de desarrollo. La recomendación debe incluir una matriz comparativa y justificar la opción final.

7. Arquitectura objetivo

- Presentation: pantallas, componentes, estado de UI.
- Application: casos de uso y orquestación.
- Domain: entidades, reglas y lógica financiera.
- Infrastructure: Firebase/API, correo, almacenamiento local, notificaciones y servicios externos.

Los módulos mínimos serán: auth, transactions, accounts, cards, categories, budgets, debts, goals, subscriptions, analytics, automation, notifications, search, settings y audit.

8. Modelo de datos mínimo

- User
- Transaction
- Account
- Card
- Category
- Budget
- Debt
- DebtPayment
- SavingsGoal
- Subscription
- Merchant
- FinancialInstitution
- AutomationRule
- EmailImport
- Notification
- UserPreference
- AuditLog

Codex debe definir campos, tipos, claves, relaciones, índices, reglas de seguridad, estrategia de versionado y criterios para evitar duplicados antes de crear colecciones/tablas definitivas.

9. Integración con Gmail

La integración con correo es un componente central, pero deberá implementarse únicamente con autorización explícita del usuario y minimización de datos. El flujo objetivo será:

1. Correo financiero recibido
2. Conector autorizado
3. Filtro de mensajes relevantes
4. Parser específico por entidad
5. Normalización
6. Detección de duplicados
7. Clasificación
8. Persistencia
9. Actualización del dashboard
10. Notificación de confirmación o revisión

Inicialmente considerar BCP, BBVA, Interbank, Scotiabank, Tarjeta Oh!, Yape y Plin. La arquitectura deberá usar adaptadores independientes por entidad y un parser genérico de respaldo. No almacenar el cuerpo completo del correo salvo necesidad estrictamente justificada.

10. Seguridad obligatoria

- Google Sign-In y correo/contraseña con verificación.
- MFA opcional.
- Biometría y PIN local en móvil.
- HTTPS/TLS obligatorio.
- Reglas de autorización por usuario en backend.
- Secrets fuera del repositorio.
- Rate limiting y protección contra abuso.
- Auditoría de acciones sensibles.
- Cierre y revocación de sesiones.
- Cifrado local de información sensible cuando corresponda.
- Validación de inputs y protección contra inyección/XSS/CSRF según plataforma.
- No registrar datos financieros sensibles completos en logs.

11. Diseño UX/UI

La identidad debe sentirse premium, calmada y profesional. Referencias conceptuales: Revolut, Monzo, N26, Apple Wallet, Linear, Stripe y Mercury, sin copiar diseños.

Paleta inicial propuesta

Token	Color
Primary	#635BFF
Secondary	#7C3AED
Accent	#14B8A6
Success	#22C55E
Warning	#F59E0B
Danger	#EF4444
Dark	#111827
Background	#F8FAFC
Surface	#FFFFFF
Text	#0F172A
Text Secondary	#64748B

Debe incluir Light, Dark y System Mode. Usar gradientes con moderación. Definir tipografía, espaciado, grid, radios, sombras, iconografía, motion, skeletons, estados vacíos y accesibilidad.

12. Navegación Web

- Inicio
- Movimientos
- Cuentas
- Tarjetas
- Presupuestos
- Deudas
- Metas
- Suscripciones
- Analítica
- Automatizaciones
- Configuración

Desktop debe usar sidebar y aprovechar ancho disponible con paneles de análisis; no debe ser una versión ampliada del móvil.

13. Navegación Mobile

- Inicio
- Movimientos
- Botón central +
- Análisis
- Perfil

El botón + permitirá registrar gasto, ingreso, transferencia, pago o deuda. Considerar bottom sheets, gestos, haptics, biometría y notificaciones push.

14. Dashboard mínimo

- Saldo total
- Ingresos del mes
- Gastos del mes
- Balance
- Ahorro
- Distribución por categoría
- Flujo de caja
- Gasto diario/semanal/mensual
- Próximos pagos
- Uso de tarjetas
- Deudas
- Metas y alertas

Filtros temporales: hoy, 7 días, 15 días, mes actual, mes anterior, 3 meses, 6 meses, 1 año y personalizado.

15. Inteligencia financiera

La IA se incorpora después de que el dato base sea confiable. Debe responder preguntas como:

- ¿Cuánto gasté este mes?
- ¿En qué categoría estoy gastando más?
- ¿Cuánto debo actualmente?
- ¿Qué tarjeta estoy usando más?
- ¿Cuánto puedo ahorrar este mes?
- ¿Cuál es mi gasto promedio diario?
- ¿Cuánto necesito ahorrar por mes para una meta?

Toda respuesta deberá incluir el periodo y los datos agregados usados para obtener la conclusión. El modelo no podrá consultar información de otro usuario.

16. Estrategia offline-first

- Persistencia local de movimientos esenciales.
- Cola de operaciones pendientes.
- IDs idempotentes.
- Sincronización automática.
- Resolución explícita de conflictos.
- Indicadores visuales de estado de sincronización.
- Pruebas de desconexión y reconexión.

17. Roadmap propuesto

Fase	Resultado
Fase 0	Descubrimiento, arquitectura, modelo de datos y seguridad.
Fase 1	Design System, navegación, shell de Web y Mobile.
Fase 2	Core financiero: auth, cuentas, categorías y movimientos.
Fase 3	Dashboard, filtros, búsqueda y exportación.
Fase 4	Tarjetas, presupuestos, metas, deudas y suscripciones.
Fase 5	Offline-first, sincronización, auditoría y recuperación.
Fase 6	Integración Gmail y parsers financieros.
Fase 7	Notificaciones y automatizaciones.

Fase 8	Analítica y proyecciones.
Fase 9	Asistente IA financiero.
Fase 10	Hardening, pruebas, CI/CD y beta privada.
Fase 11	Publicación estable y observabilidad.

18. Definición del MVP

- Registro/login seguro.
- Dashboard básico.
- Movimientos manuales.
- Cuentas y categorías.
- Tarjetas con saldo/línea/fechas.
- Presupuesto mensual.
- Búsqueda y filtros.
- Exportación CSV.
- Offline y sincronización.
- Primer conector Gmail para al menos una entidad bancaria.
- Auditoría mínima y gestión de sesión.

No incluir en el MVP: IA avanzada, múltiples bancos simultáneos, inversiones complejas, recomendaciones automáticas de crédito, open banking no disponible oficialmente o integraciones que requieran compartir credenciales bancarias.

19. Testing requerido

- Unit tests para reglas financieras y parsers.
- Integration tests para auth, backend, Gmail y sincronización.
- E2E para flujos críticos Web y Mobile.
- Security tests de permisos y aislamiento por usuario.
- Pruebas de duplicados e idempotencia.
- Pruebas de fechas, zonas horarias y monedas.
- Pruebas offline/reconexión.
- Pruebas de accesibilidad y responsive.
- Pruebas de rendimiento en listas y dashboard.

20. CI/CD y GitHub

Crear un repositorio NUEVO y PRIVADO. Estructura sugerida:

- README.md
- ARCHITECTURE.md
- ROADMAP.md

- SECURITY.md
- CONTRIBUTING.md
- CHANGELOG.md
- docs/
- apps/web/
- apps/mobile/
- packages/shared/
- backend/ o functions/
- infra/
- .github/workflows/

Flujo sugerido: main protegida, develop opcional, ramas feature/* y fix/*, pull requests, Conventional Commits, lint, tests, build, análisis de seguridad, staging y producción.

21. Entregables que Codex debe producir antes de construir el producto completo

11. Resumen ejecutivo
12. Objetivo general y específicos
13. Alcance / fuera de alcance
14. Arquitectura propuesta
15. Matriz tecnológica y recomendación final
16. Modelo de datos
17. Arquitectura de seguridad
18. Arquitectura de Gmail
19. Arquitectura Web
20. Arquitectura Mobile
21. Design System
22. Wireframes textuales
23. Roadmap
24. Backlog priorizado
25. Historias de usuario
26. Criterios de aceptación
27. Riesgos y mitigaciones
28. Estrategia de testing
29. CI/CD
30. Definición exacta de MVP, V2 y V3
31. Estructura del repositorio
32. Plan de implementación por PRs pequeños

22. Prompt maestro listo para pegar en ChatGPT/Codex

Actúa como Product Manager, Arquitecto de Software Senior, Diseñador UX/UI FinTech, Especialista en Seguridad y Desarrollador Full Stack Senior.

Quiero crear desde cero un proyecto NUEVO e INDEPENDIENTE de finanzas personales, sin modificar ni reutilizar automáticamente ningún repositorio previo. La solución debe funcionar en Web, PWA, Android e iOS.

OBJETIVO: construir una plataforma personal que centralice gastos, ingresos, cuentas, tarjetas, deudas, presupuestos, metas, suscripciones y analítica, reduciendo al mínimo el ingreso manual mediante integraciones autorizadas como Gmail. Debe ser minimalista, premium, rápida, offline-first, segura y escalable.

ANTES DE PROGRAMAR: no escribas el producto completo. Primero realiza discovery técnico, arquitectura, modelo de datos, seguridad, UX/UI, comparación de tecnologías y definición del MVP. Propón alternativas y toma una decisión justificada.

REQUISITOS CLAVE:

- Dashboard con saldo, ingresos, gastos, balance, ahorro, categorías, flujo de caja, tarjetas, deudas y próximos pagos.
- Movimientos con búsqueda, filtros y categorización.
- Cuentas, tarjetas, presupuestos, metas, deudas y suscripciones.
- Integración Gmail mediante OAuth/autorización explícita y parsers por entidad bancaria.
- Detección de duplicados e idempotencia.
- Offline-first y sincronización segura.
- Light/Dark/System Mode.
- UX diferenciada para desktop y móvil.
- Biometría/PIN en móvil cuando corresponda.
- Exportación CSV/Excel/PDF.
- IA financiera solo después de tener datos confiables y siempre explicable.
- Seguridad por usuario, secretos fuera del repositorio, auditoría y revocación de sesiones.
- Nunca guardar PIN, CVV, contraseñas bancarias o credenciales bancarias.

EVALÚA: Angular + Flutter; Angular + Ionic/Capacitor; Flutter completo. Crea una matriz y recomienda una opción.

DISEÑO: identidad propia inspirada conceptualmente en Revolut, Monzo, N26, Apple Wallet, Linear, Stripe y Mercury, sin copiar. Paleta inicial: #635BFF, #7C3AED, #14B8A6, #22C55E, #F59E0B, #EF4444, #111827, #F8FAFC, #FFFFFF, #0F172A y #64748B.

ARQUITECTURA: separar presentation, application, domain e infrastructure. Módulos: auth, transactions, accounts, cards, categories, budgets, debts, goals, subscriptions, analytics, automation, notifications, search, settings y audit.

ENTREGABLES INICIALES: resumen ejecutivo, objetivos, alcance, arquitectura, recomendación tecnológica, modelo de datos, seguridad, arquitectura Gmail, arquitectura Web/Mobile, Design System, wireframes textuales, roadmap, backlog, historias de usuario, criterios de aceptación, riesgos, testing, CI/CD, MVP/V2/V3 y estructura del repositorio.

Después de mi revisión, implementa por fases y PRs pequeños. Cada fase debe incluir código, tests, documentación, criterios de aceptación, riesgos y pasos de validación. No pases a una fase nueva si la anterior no compila, no pasa tests o tiene problemas críticos de seguridad.

23. Criterio de éxito

El proyecto se considerará correctamente encaminado cuando exista un repositorio privado nuevo, una arquitectura aprobada, un MVP claramente limitado, pruebas automáticas para funciones críticas, aislamiento real de datos por usuario, sincronización confiable y una experiencia coherente entre Web y Mobile. La prioridad es confiabilidad financiera y seguridad antes que cantidad de funcionalidades.